Cryptologie Avancée (CRYPTA / MU5IN951) : examen réparti n°2

documents et calculatrice autorisés / durée 2h / 3 pages

6 février 2020

La société Snake Oil Inc. a soumis un nouvel algorithme de chiffrement à une Agence d'évaluation pour le compte de laquelle vous êtes expert. Vous devez décider s'il est sûr ou pas.

Vous connaissez la réputation sulfureuse de l'entreprise qui soumet le dossier, et vous savez que les ingénieurs de chez Snake Oil Inc. racontent n'importe quoi en général. Mais comme vous travaillez dans une Agence Sérieuse, vous devez argumenter point par point votre refus de délivrer une certification de sécurité à l'algorithme soumis par Snake Oil Inc.

La spécification complète du mécanisme soumis à votre évaluation est en annexe.

Dossier d'évaluation

Il s'agit de démonter point par point les affirmations de Snake Oil Inc. dans la partie « promesses ».

Problèmes des primitives

Question 1 : Démontrez que la Snake Oil PRF n'est pas une fonction pseudo-aléatoire cryptographiquement sûre (la promesse n°2 est fausse). Indice : une seule requête suffit.

Question 2 : Démontrez que même si on avait définit la Snake Oil PRF dans « l'autre sens » (avec $x\|i$ comme graine du PRG), elle ne serait pas non plus cryptographiquement sûre.

Question 3 : Démontrez que le Snake Oil PRG n'est pas un générateur pseudo-aléatoire cryptographiquement sûr (la promesse n°1 est fausse).

Problèmes des modes opératoires de chiffrement

Question 4 : Démontrez que si $\mathcal{G}$ est un PRG cryptographiquement sûr, alors l'algorithme suivant :

$$\mathcal{G}'(x_1 \dots x_n x_{n+1} \dots x_{2n}) := \mathcal{G}(x_{n+1} \dots x_{2n})$$

est lui aussi un PRG cryptographiquement sûr ($\mathcal{G}'$ fonctionne en lançant $\mathcal{G}$ mais en ignorant la première moitié de la graine).

Question 5 : En utilisant le résultat question précédente, démontrez que l'affirmation de Snake Oil Inc. sur la fiabilité du mode opératoire de chiffrement proposé (la promesse n°3) est fausse.

Question 6 : Démontrez que SO-Enc (c.a.d. la combinaison du mode opératoire Snake Oil Symmetric Encryption avec le Snake Oil PRG n'offre pas la sécurité sémantique sous attaques à clairs choisis (la promesse n°4 est fausse).

Problèmes des modes opératoires d'authentification

Question 7 : Démontrez qu'il existe un chiffrement $\mathcal{E}$ qui est IND-CPA et un MAC $\mathcal{M}$ qui est existentiellement inforgeable tels que le Snake Oil Authenticated Encryption construit avec $\mathcal{E}$ et $\mathcal{M}$ n'offre pas le chiffrement authentifié (la promesse n°5 est fausse).

Question 8 : Démontrez que le Snake Oil Mac n'est pas sûr, autrement dit qu'il n'est pas existentiellement inforgeable (la promesse n°6 est fausse).

Indice : si t est le tag d'un message M, que se passe-t-il si on calcule le tag de $M\|t \oplus x$?

Question 9 : En vous inspirant du mode opératoire de Merkle-Damgård, proposez une contre-mesure qui évite le problème de la question précédente.

Spécification

Le symbole ε désigne la chaine de bits vide, et $\|$ désigne l'opérateur de concatenation des chaines de bits.

Primitives

Core Le Snake Oil Core est une fonction $C : \{0,1\}^* \to \{0,1\}$ qui fonctionne comme ceci :

- Par définition, $C(\varepsilon) = 0, C(0) = 1$ et $C(1) = 0$.
- Si l'entrée est de taille supérieure ou égale à 2, on applique l'algorithme suivant pour calculer $C(x)$:
  1. Chercher le plus grand suffixe de x qui apparaît déjà avant (appelons-le s).
  2. Si un tel suffixe n'existe pas, alors $C(x) = 0$.
  3. Repérer l'avant-dernière occurence de s dans x (la dernière, c'est le suffixe lui-même), et appeler b le bit qui suit.
  4. La valeur de $C(x)$ est $1 - b$.

Par exemple (le suffixe est surligné, son occurrence précédente est soulignée, b est en gras) :

$$C(0101000\underline{10011}\mathbf{0}0010100001\overline{010011}) = 1$$
$$C(1010101010\underline{00011}\mathbf{1}000000\overline{00011}) = 0$$
$$C(\underline{1\overline{111111}}\mathbf{1}) = 0$$
$$C(11111111\overline{0}) = 0$$

Générateur pseudo-aléatoire Le Snake Oil PRG, fonctionne en itérant le Core à partir de la graine. Plus précisément, à partir d'une graine `seed` (de taille arbitraire), il fonctionne de la façon suivante :

$$x_0 \leftarrow \texttt{seed} \qquad\qquad y_0 \leftarrow \varepsilon$$
$$x_{i+1} \leftarrow x_i \| \mathsf{Core}(x_i) \qquad\qquad y_{i+1} \leftarrow y_i \| \mathsf{Core}(x_i)$$

La valeur renvoyée est finalement $\mathsf{PRG}(\texttt{seed}, 1^t) := y_t$.

Par exemple, en partant de 010, on trouve :

$$x_0 = \underline{0}1\overline{0}$$
$$x_1 = 01\underline{0}\overline{0}$$
$$x_2 = \underline{01}0\overline{01}$$
$$x_3 = 0100\underline{1}\overline{1}$$
$$x_4 = 0\underline{10}01\overline{10}$$
$$x_5 = 010\underline{01}1\overline{01}$$
$$x_6 = \underline{010}011\overline{010}$$
$$x_7 = 01001\underline{10}\overline{101}$$
$$x_8 = 010\underline{011}01\overline{011}$$
$$\vdots$$
$$x_i = 0100110101110001000011\underline{1011001}0100100111010001100000101101111100110011111101010\overline{1011001}\ldots$$

Par conséquent, $\mathsf{PRG}(010) = 0110101110001000011110110010100100111010001100000101101111100 1\ldots$.

Fonction pseudo-aléatoire La Snake Oil PRF est définie de la façon suivante. Pour générer un indice de la PRF offrant n bits de sécurité, on génère une chaine aléatoire de n bits. Pour évaluer la PRF sur une entrée $x \in \{0,1\}^n$ avec un indice $i \in \{0,1\}^n$, on produit n bits avec le Snake Oil PRG à partir de la graine $i\|x$. Autrement dit : $V(i,x) := \mathsf{PRG}(i\|x, 1^n)$.

Modes opératoires

Chiffrement Le Snake Oil Symetric Encryption Scheme réalise du chiffrement à partir de n'importe quel générateur pseudo-aléatoire $\mathcal{G}$. Pour chiffrer un message M (de taille arbitraire) avec une clef K de n bits, on choisit aléatoirement $IV \in \{0,1\}^n$ et on forme une graine de $2n$ bits en concaténant la clef et l'IV. On génère un masque x de la même taille que M avec $x \leftarrow \mathcal{G}\left(K\|IV, 1^{|M|}\right)$. Enfin, le chiffré est $(IV, x \oplus M)$.

Lorsque cette technique est utilisée avec le Snake Oil PRG, on note l'algorithme de chiffrement résultant SO-Enc.

Code d'authentification de message Le Snake Oil MAC fonctionne de la façon suivante, avec le paramètre de sécurité n. Il utilise une PRF quelconque, décrite par les deux algorithmes (I, V) où I génère un indice aléatoire et V évalue la PRF.

— Pour générer une clef du MAC, générer un indice k de la PRF.
— Pour générer le tag (de n bits) à partir d'un message M de taille arbitraire, ajouter un bit à 1 à la fin, puis autant de bits à 0 que nécessaire pour que la taille (en bits) devienne un multiple de n. Découper alors le message en blocs de n bits : $M = M_0 \| M_1 \| \ldots \| M_\ell$, puis calculer :

$$x_0 \leftarrow 00000\ldots0$$
$$x_{i+1} \leftarrow V(k, M_i \oplus x_i)$$

Le tag est $x_{\ell+1}$.

Lorsque la PRF utilisée est la Snake Oil PRF, alors ce MAC est noté SO-MAC.

Chiffrement authentifié Le Snake Oil Authenticated Encryption repose sur i) un chiffrement IND-CPA $\mathcal{E}$, et ii) un MAC $\mathcal{M}$. Pour effectuer le chiffrement authentifié d'un message M, avec une clef de chiffrement K_e et une clef d'authentification K_a, on renvoie la paire $(\mathcal{E}_{K_e}(M), \mathcal{M}_{K_a}(M))$.

Promesses

La société Snake Oil Inc. prétend les choses suivantes :

1. Le Snake Oil PRG est un générateur pseudo-aléatoire cryptographiquement sûr.
2. La Snake Oil PRF est une fonction pseudo-aléatoire cryptographiquement sûre.
3. Si $\mathcal{G}$ est un PRG cryptographiquement sûr, alors le mode opératoire « Snake Oil Encryption Scheme » est sémantiquement sûr sous des attaques à clairs choisis (IND-CPA).
4. SO-Enc est un mécanisme de chiffrement symétrique sémantiquement sûr (IND-CPA).
5. Si $\mathcal{E}$ est un chiffrement IND-CPA et $\mathcal{M}$ est un MAC sûr, alors le mode opératoire « Snake Oil Authenticated Encryption » garantit le chiffrement authentifié.
6. Le Snake Oil MAC est un code d'authentification de message cryptographiquement sûr (existentiellement inforgeable).

Correction

1 On remarque que le premier bit de la sortie est $\mathsf{Core}(i\|x)$.

Il suffit de faire *une* requête avec l'argument $x = 00000\ldots0$. Quelle que soit la valeur de l'indice, alors le premier bit de la sortie vaut 1. Ce comportement n'est pas aléatoire ! Si on avait affaire à une fonction aléatoire, on devrait avoir ceci dans 50% des cas seulement. Par conséquent, renvoyer le premier bit de la sortie donne un distingueur qui a un avantage de 1/2, ce qui n'est pas négligeable.

2 Dans le fond, c'est le problème inverse : $\mathsf{Core}(i\|x)$ ne dépend probablement pas de i, mais $\mathsf{Core}(x\|i)$ ne dépend probablement pas de x !

Du coup, le distingueur fonctionne de la façon suivante : effectuer une requête avec $x = 0000\ldots0$ puis une requête avec $x = 10000\ldots0$. Si la réponse est la même, répondre « PRF », sinon répondre « aléatoire ».

Quelle est la probabilité de répondre « aléatoire » par erreur ? Pour que $\mathsf{Core}(000\ldots x\|i) \neq \mathsf{Core}(100\ldots x\|i)$, il faudrait que $i = 1000\ldots0$ ou bien $i = 0000\ldots0$. Or, vu le choix aléatoire de i, ces deux évènements ne peuvent survenir qu'avec probabilité 2^{-n} chacun, ce qui est négligeable.

Si le distingueur est confronté à une fonction « vraiment aléatoire », il va répondre « aléatoire » avec probablité 1/2, donc il a un avantage de 1/2, ce qui n'est pas non plus négligeable.

3 Cette fois-ci, la « graine » n'est pas du tout sous le contrôle de l'adversaire. L'argument principal est grosso-modo le suivant : le premier bit du flux pseudo-aléatoire dépend entièrement de la graine, qui est aléatoire et secrète, donc on ne va pas réussir à le prédire. Mais le 100000ème bit du flux pseudo-aléatoire, lui, est très probablement entièrement déterminé par les 99999èmes premiers bits du flux, qui sont connus, eux !

On construit donc un distingueur qui casse le Snake Oil PRG de la façon suivante : i) si n est le paramètre de sécurité, observer $n^2 + 1$ bits du flux pseudo-aléatoire. On les note $x_0, \ldots, x_{n^2}$. Si $x_{n^2} = \mathsf{Core}(x_0, \ldots, x_{n^2-1})$, alors répondre « PRG » sinon répondre « aléatoire ».

Le problème, c'est qu'il n'est pas très simple d'estimer précisément l'avantage de ce distingueur. Mais on admet qu'il qu'il est élevé !

4 Contraposée : on démontre que si $\mathcal{G}'$ est cassé, alors $\mathcal{G}$ est cassé. Supposons qu'un algorithme polynomial A distingue la sortie de $\mathcal{G}'$ d'un flux de bits « vraiment aléatoires ». Autrement dit, A a un comportement différent selon qu'on lui présente la sortie de $\mathcal{G}'$ (avec une graine de $2n$ bits aléatoires) ou bien des bits uniformément distribués. Mais en fait, la sortie de $\mathcal{G}'$, c'est la sortie de $\mathcal{G}$ (avec une graine de n bits aléatoires). Donc l'algorithme A, sans modification, agit comme un distingueur pour $\mathcal{G}$, et « casse » ce dernier. CQFD.

5 Le fond de l'affaire, c'est que si $\mathcal{G}$ est un PRG sûr, alors on sait que sa sortie « à l'air aléatoire » tant que la graine utilisée est choisie aléatoriement, et même si l'expérience est répétée plusieurs fois. Mais par contre, on ne possède pas de garanties très claires sur ce qui se passe si on utilise plusieurs graines *correlées*.

En l'occurrence, le mode opératoire de chiffrement décrit par Snake Oil Inc. est *peut-être* sûr : si le PRG $\mathcal{G}$ se comporte à peu près comme une fonction aléatoire, alors tout irait bien.

Mais on voit bien qu'avec le $\mathcal{G}'$ de la question précédente, *qui est bien un PRG sûr*, le chiffrement n'offre aucune forme de sécurité : le masque généré par $\mathcal{G}'$ ne dépend pas de K mais seulement de IV (puisque G' ignore la première moitié de sa graine). Or IV fait partie du chiffré, donc on peut déchiffrer... même sans connaître K.

6 Dans le cours, on a vu que si le PRG est sûr, alors le chiffrement qui consiste à générer un masque avec le PRG puis à le XORer sur le clair est IND-CPA. Ici, en fait, c'est l'inverse : on va voir que si le PRG est cassé, alors le chiffrement ne peut pas être IND-CPA.

Supposons qu'un adversaire $\mathcal{A}$ efficace capable de distinguer la sortie du PRG de bits aléatoires existe. On construit un adversaire qui casse la sécurité sémantique du chiffrement. Cet adversaire choisit un message M_0 complètement aléatoire, et choisit $M_1 = 000\ldots0$ (de la bonne taille). Il récupère le chiffré C du challenger, puis lance le distingueur $\mathcal{A}$ dessus, et renvoie ce que $\mathcal{A}$ a renvoyé.

Si le challengeur a renvoyé un chiffré de M_1, alors on fournit directement la sortie du PRG au distingueur. Si Le challengeur a renvoyé un chiffré de M_0, alors on fourni du charabia uniformément aléatoire au distingueur. Et comme le distingueur est capable de faire son boulot, il va avoir un comportement différent dans les deux situations.

Par conséquent, ceci casse la sécurité sémantique du chiffrement : on est capable de se comporter différemment sur les chiffrés de M_0 et sur les chiffrés de M_1.

7 En cours, on a vu que « encrypt-then-MAC » était sûr. Mais ici, la Snake Oil Inc. fait du « MAC-then-encrypt » ! Et le problème, c'est que le MAC peut faire fuir de l'information, ce qui va casser la sécurité sémantique de l'ensemble.

Par exemple, si le MAC est déterministe, c'est la catastrophe : dans le jeu qui définit la notion IND-CPA, l'adversaire peut choisir deux messages quelconques puis demander le chiffrement de M_0. Il apprend alors *le* tag de M_0. Il suffit de regarder si le « défi » envoyé par le challengeur contient le tag de M_0 ou bien autre chose.

En fait, il y a des problèmes y compris avec des MACs randomisés. Plus précisément, soient $\mathcal{E}$ et $\mathcal{M}$ qui satisfont les conditions de la question (sûrs tous les deux). On fabrique un autre MAC $\mathcal{M}'$ qui fonctionne en utilisant $\mathcal{M}$ pour calculer le tag, puis rajoute le premier bit du message à la fin du tag (c'est ça la fuite d'information).

On prétend que $\mathcal{M}'$ est toujours un MAC sûr. En effet, un adversaire capable de forger une paire (message, tag) valide pour $\mathcal{M}'$ peut aisément être converti en un adversaire qui forge $\mathcal{M}$: il suffit de retirer le bit excédentaire, et on obtient une paire message-tag valide pour $\mathcal{M}$.

Et le problème, c'est qu'avec la combinaison $(\mathcal{E}, \mathcal{M}')$ —qui satisfait les bonnes hypothèses— il est très facile de casser la sécurité sémantique. Il suffit que l'adversaire soumette au challenger deux messages qui diffèrent sur le premier bit. Même si $\mathcal{E}$ ne laisse rien fuiter, $\mathcal{M}'$ révèle le premier bit du clair. Et ceci suffit à savoir à coup sûr si on a affaire à un chiffrement du premier message ou du deuxième.

8 On décrit ici un adversaire qui forge le MAC. Il choisit deux messages M et M' de la même taille, et on les soumet à l'oracle de signature auquel il a accès. Il obtient les deux tags t et t'. On découpe le message M' en blocs de n bits (avec le bourrage) : $M' = M'_1 \| M'_2 \| \ \ldots$.

L'adversaire pose $M'' := M \| t \oplus M'_1 \| M'_2 \| M'_3 \| \ldots$, puis émet la paire (M'', t'). On prétend qu'il forge le MAC avec probabilité 1.

En effet, il se trouve que t' est aussi le tag du message M' ! En effet, au moment où le traitement du bloc $t \oplus M'_1$ va commencer, la valeur de x_i est t (le tag de M). Et du coup, calculer le MAC de M' avec $x_i = 0$ ou de $M' \oplus t$ avec $x_i = t$ revient exactement au même.

Cette attaque sur CBC-MAC est un grand classique.

9 A priori, ajouter la taille du message dans le bourrage empêche ce problème.